

P1281R0

Feature Presentation

Published Proposal, 2018-10-07

This version:

<https://wg21.link/p1281r0>

Author:

[Isabella Muerte](#)

Audience:

SG15

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1281R0](#)

Current Source:

[slurps-mad-rips/papers/proposals/feature-presentation.bs](https://ericniebler.com/2018/10/07/slurps-mad-rips/papers/proposals/feature-presentation.bs)

Abstract

`if constexpr` does not work at any scope beyond that of a function. Let's add an attribute and function so that we can have feature or platform specific code without having to use the preprocessor.

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design

4 Example

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

Languages like Rust or D or dynamically typed languages like Python and Ruby have some form of a global "static" if (static in the sense that it executes at compile time). While a previous attempt was made to add this to C++, it was later reduced down to `if constexpr`. However, `if constexpr` is limited in that it still requires code be semantically correct. This makes it very difficult to use for wrapping platform specific code for cross platform interfaces. Instead, we must still rely on preprocessor `ifdefs` in order to get what we want. In Rust, they have a `#[cfg()]` attribute that allows specific declarations and definitions to be removed from the AST at compile time. This paper proposes to add both a `[[feature()]]` attribute as well as a function `std::feature` that would permit removing the need for `ifdef` at any non-function level scope. This is desired due to programmers having to target a platform, and in some cases multiple platforms that behave more or less the same, but have different identifiers available.

§ 3. Design

The design of the feature presentation is two-fold. When a compiler sees the `[[feature("key")]]` attribute, it has to see if it is in the allowed or blocked list of "features". If any of the `key` strings are allowed, the compiler parses as normal. In the event that the key is absent or in an explicit block list, the compiler attempts to treat the declaration and all its contents as a token soup, thus removing it from the AST. In other words, the code contained within a declaration where the feature attribute key is disallowed must be syntactically, but not semantically, correct. If the feature key given is allowed, then the compiler treats the declaration like normal.

We also provide a `std::feature()` function which allows users to check whether a given set of strings are valid inside of `if constexpr`.

Lastly, this attribute does not interfere with ODR, since two translation units compiled with the same named definition but different features is not permitted.

§ 4. Example

As a example, let's look at some simple interface code that could be written with this attribute, where we are targeting both DirectX and Vulkan:

```
struct [[feature("vulkan")]] Device {
    [[feature("glsl-to-spirv")]]
    static Shader compile(std::filesystem::path filename);
    static Shader load (std::filesystem::path spirv_file);
};

struct [[feature("direct-x")]] Device {
    [[feature("hlsl-to-spirv")]]
    static Shader compile (std::filesystem::path filename);
    static Shader load (std::filesystem::path spirv_file);
};
```

In the above sample, we have two interfaces for a class named `Device`. In this case, we might want to have the ability to compile shaders to an intermediate SPIR-V representation before we release the product that uses this code. However, allowing users to use the product's compiler might not be desired and thus we can choose to not only switch between these operations but to disable the ability to even call `Device::compile` in all interfaces.